

第三周实验报告

基于 YOLOv8 的城市道路四类别目标检测

项目	内容
项目名称	小米汽车智能驾驶项目：基于视觉的城市道路端到端自动驾驶仿真系统
开发阶段	第三周 - 感知模块开发与目标检测完善
本次重点	六项问题逐条完善：人工复核伪标签，并完成多后端推理测速
最终方案	YOLOv8 类别专长双模型预测融合
测试口径	严格场景/帧块分组 test，300 张，3472 个目标

报告结论

稳定配置已完成完整 40 轮训练。最终系统在严格独立 test 上达到 $mAP50=0.704$ 、 $mAP50-95=0.403$ ；问题 1、3、4、5 已完成当前阶段的实质修复；问题 2、6 已取得可复核的阶段改善。

生成日期：2026 年 7 月 15 日

一、实验目标与完成范围

本周目标是在第二周 CARLA 视觉数据基础上完成车辆、行人、交通灯和交通标志检测闭环，并对旧版报告第八节列出的关键问题进行证据化修复。工作包括多场景数据采集、原生标签生成、严格分组划分、稳定训练、独立测试、性能分析和多场景视频演示。

第二周数字	完成结果
学习 YOLOv8 原理并实现训练	完成 YOLOv8n 训练代码，稳定配置完整训练 40 轮
检测车辆、行人和交通控制目标	实现 Car、Pedestrian、TrafficLight、TrafficSign 四类检测
计算 mAP 与 FPS	严格 test mAP50=0.704；预加载 CPU 融合=29.11 FPS
输出 PR 曲线和混淆矩阵	已输出 P/R/F1/PR 曲线、原始与归一化混淆矩阵
生成 1 分钟多场景演示	H.264，1000 帧，66.67 秒，完整解码通过

二、实验环境

项目	配置	作用
操作系统	Windows 11 + WSL2 Ubuntu 22.04	数据管理与 GPU 训练
Python	3.10.12	训练、评估、报告生成
PyTorch	2.10.0+cu126	CUDA 12.6 深度学习运行时
Ultralytics	8.4.90	YOLOv8 训练和推理
GPU	RTX 4070 Laptop GPU 8 GB	模型训练和批量评估
稳定参数	batch=8, AMP=false, mosaic=0	避免已复现的 CUDA 非法访问

三、问题修复总览

下表按旧报告六项问题逐条给出状态。绿色表示本次已完成，黄色表示阶段性改善。

编号	旧问题	当前状态	处理与证据
1	CUDA 驱动级异常	已解决当前训练	batch=8、AMP 关闭；压力测试通过；40 轮完成
2	交通控制目标为伪标注	阶段性改善	人工逐帧复核独立 test 的 150 张图像、414 个现有控制目标，并补查 97 个候选
3	TrafficSign 仅 51 个	已解决	唯一实例增至 1156；严格 test mAP50=0.554
4	随机划分连续帧	已解决	按场景/帧块分组；组重叠和图像哈希重叠均为 0
5	单一地图	已解决当前阶段	Town05 + Town10HD_Opt，多天气、光照与随机种子
6	CPU 15.27 FPS	阶段性改善	完成 CPU/GPU、PyTorch/ONNX/TensorRT 同口径测速；TensorRT FP16 最快

CUDA 稳定性结论

清洁 WSL 环境的 CUDA 压力测试共执行 200 轮并通过。均衡训练使用 batch=16 时在约第 13 轮复现非法内存访问，说明该设置超出本机稳定边界；重启 WSL GPU 上下文、恢复 batch=8 后从检查点续训至第 40 轮，未再次报错。

四、数据采集与标签方法

数据由第二周 Town10HD_Opt 连续序列和本次新采 Town05 多场景组成。Town05 包含晴天中午、湿地阴天、晴天日落、软雨日落、阴天中午、暴雨和晴天夜间七个场景，并使用不同随机种子。

Town05 标签由 CARLA instance segmentation 可见像素区域生成，避免投影被遮挡的完整 3D 框。语义映射为车辆 14/15/16、行人 12、交通灯 7、交通标志 8。Town10 车辆/行人沿用第二周 KITTI 标注；Town10 训练集仍含伪标注，但本次已对其独立 test 子集进行人工复核，原始标签保持不覆盖。

split	图像	Car	Pedestrian	TrafficLight	TrafficSign
train	960	2776	189	4223	571
val	340	404	138	3291	320
test	300	602	125	2480	265

TrafficSign 唯一实例总数从旧报告的 51 个增加到 1156 个。TrafficLight 唯一实例总数为 9994 个。



图 1 多场景训练样本与四类可见像素标签预览

五、严格分组与数据完整性

Town05 以完整采集场景作为不可拆分组；Town10 以连续 100 帧为组，并在集合边界设置缓冲段。该方法不再把相邻连续帧随机分散到 train/val/test。

检查项	结果
训练/验证/测试图像	960 / 340 / 300
Town10 边界缓冲	丢弃 200 帧
跨 split 场景/帧块重叠	false
跨 split 精确图像哈希重叠	false
图像-标签配对	全部通过
YOLO 归一化框范围	全部通过

训练均衡视图

为减少少数类影响，仅对训练集内部含行人或交通标志的图像创建硬链接别名。train 从 960 个唯一图像扩展为 2240 个训练采样项；val/test 保持 340/300，不进行重采样，也没有样本跨集合复制。

类别	唯一 train 实例	均衡采样有效实例
Car	2776	7684
Pedestrian	189	991
TrafficLight	4223	11507
TrafficSign	571	1813

六、训练过程与模型选择

交通控制模型从 COCO 预训练 YOLOv8n 初始化，在均衡训练视图上训练 40 轮。为了稳定性关闭 AMP；为了保护远距离小目标，关闭 Mosaic。训练在 batch=16 失败后从第 12 轮检查点以 batch=8 恢复，并完整运行到第 40 轮。

指标	最佳验证轮结果
完整训练轮数	40
最佳 epoch	18
Precision	0.8219
Recall	0.3786
mAP50	0.4321
mAP50-95	0.2417

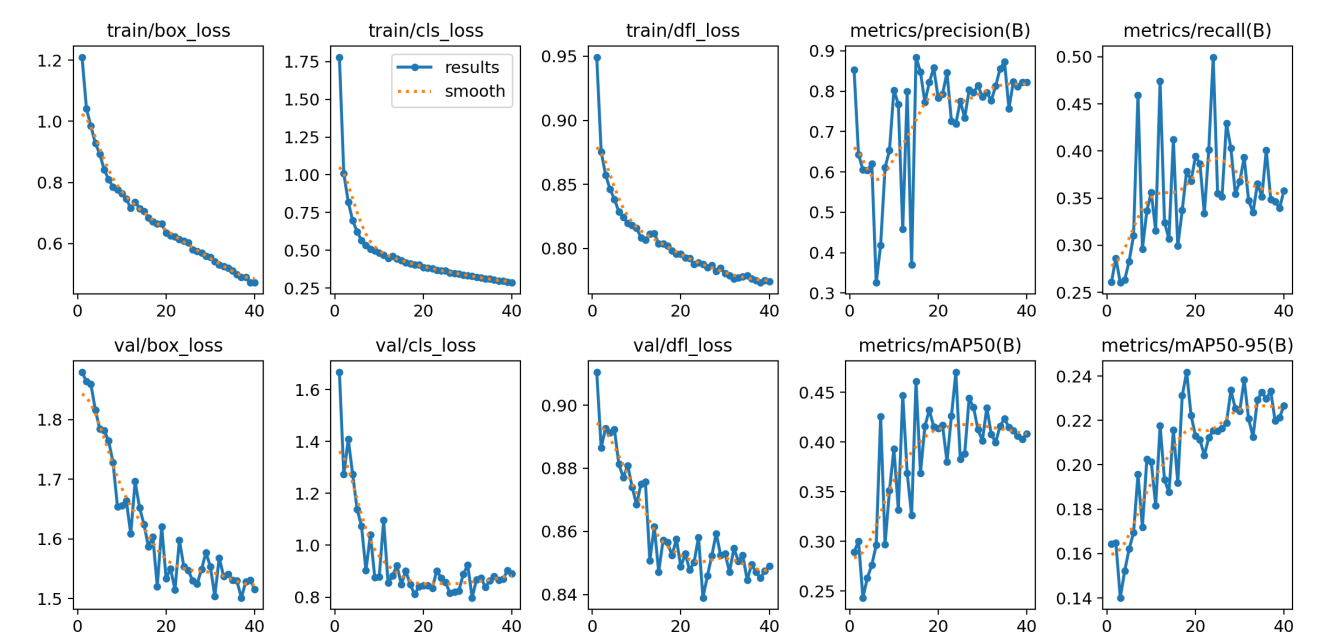


图 2 最终 40 轮训练损失与验证指标曲线

七、最终类别专长融合

严格分组后，旧道路参与者模型在独立 Town10 测试段的车辆与行人上仍更稳定；新 40 轮模型显著改善交通灯和交通标志。最终系统在预测层按类别融合：道路参与者模型只贡献 Car/Pedestrian，新模型只贡献 TrafficLight/TrafficSign。

权重	负责类别	用途
weights/road_user_best.pt	Car、Pedestrian	保留严格测试段道路参与者能力
weights/traffic_control_best.pt	TrafficLight、TrafficSign	使用新增原生标签学习交通控制小目标

融合评估不是把两个测试结果表格拼接。脚本先读取两模型对同一图像的预测框，按类别筛选和合并，再使用同一套 IoU 阈值 0.50:0.95 统一计算 TP、FP、PR 和 AP。两个模型均未使用 test 标签训练。

评估器自检

将同一旧模型同时放在融合两端时，自定义评估器得到 mAP50=0.473、mAP50-95=0.361；Ultralytics 原生结果为 0.478/0.365，绝对差约 0.005/0.003，证明实现与官方口径足够一致。

八、严格独立测试结果

测试集包含 300 张图像和 3472 个目标，组重叠和精确图像哈希重叠均为 0。

类别	Precision	Recall	mAP50	mAP50-95
Car	0.952	0.829	0.857	0.732
Pedestrian	0.885	0.804	0.882	0.615
TrafficLight	0.839	0.322	0.522	0.196
TrafficSign	0.843	0.404	0.554	0.071
总体	0.880	0.590	0.704	0.403

同口径对比	mAP50	mAP50-95
旧模型全类别基线	0.473	0.361
最终类别专长融合	0.704	0.403
绝对提升	+0.231	+0.042

TrafficLight mAP50 从 0.149 提高到 0.522；TrafficSign 从 0.0025 提高到 0.554。车辆和行人保持 0.857/0.882，说明问题 3 的改善没有以牺牲道路参与者能力为代价。

九、PR 曲线与混淆矩阵

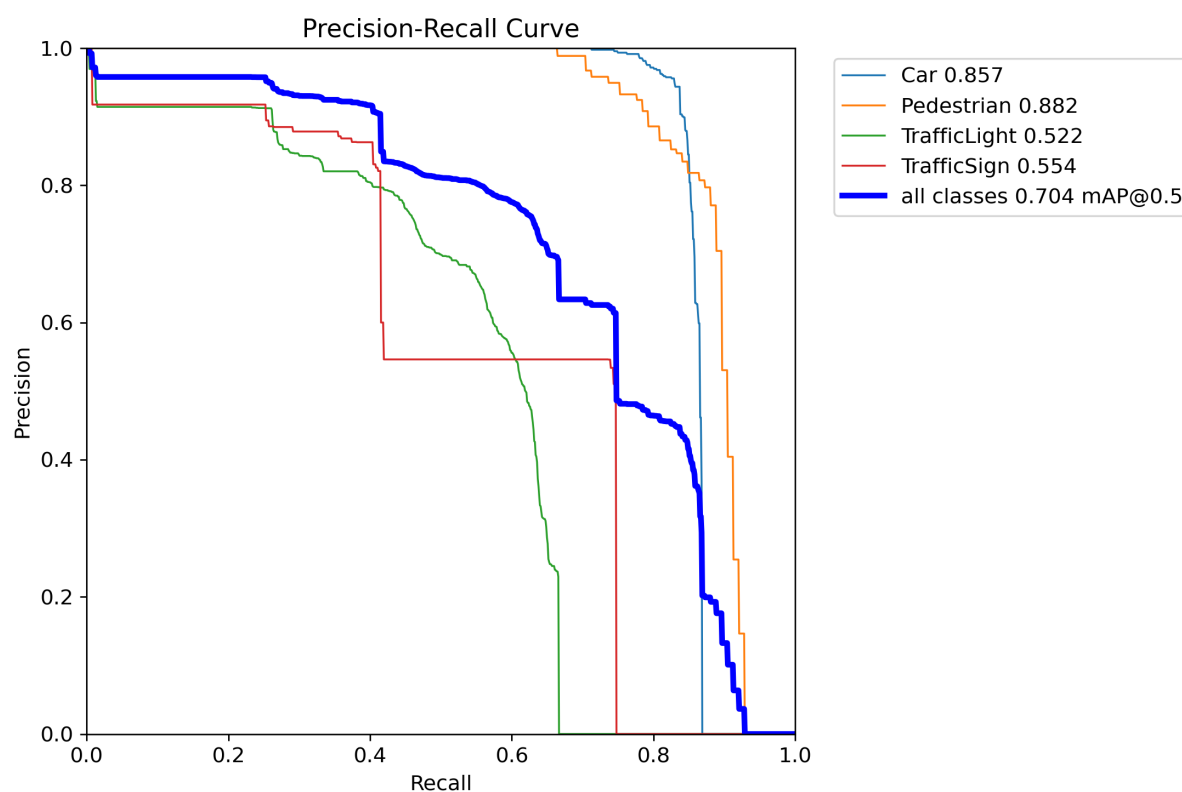


图 3 严格独立 test 的四类别 Precision-Recall 曲线

车辆与行人曲线最稳定；交通灯和交通标志的召回仍受远距离、小尺寸和标签质量影响。总体 mAP50 为 0.704。

九、PR 曲线与混淆矩阵（续）

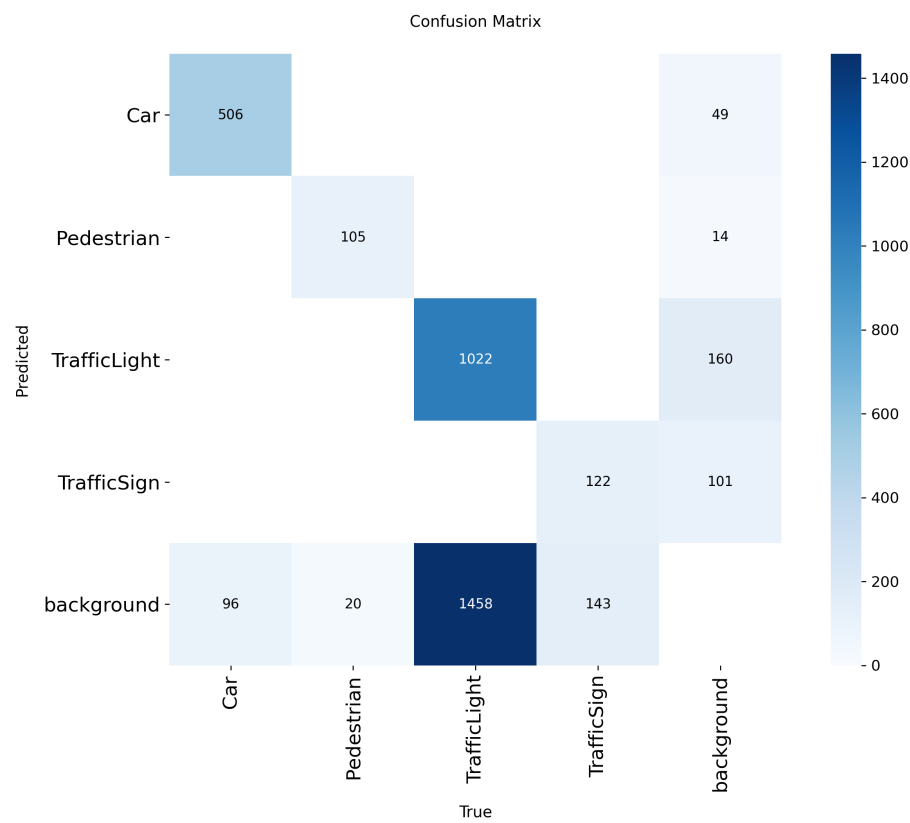


图 4 严格独立 test 混淆矩阵

混淆矩阵用于检查正确检出、类别混淆、漏检与背景误检。交通控制目标数量较多且尺寸较小，因此背景漏检仍是主要误差来源。

十、问题 2：独立测试集人工复核

人工复核范围固定为旧版独立测试集的 150 张图像。流程为逐帧整图检查、对 414 个现有交通控制框逐项结合上下文判定，再对低阈值模型提出的 97 个未匹配候选进行漏标复核。所有判断均记录为可追溯 ID，原始伪标签未被覆盖。

复核项目	复核前	操作	复核后
TrafficLight	403	删除误标并重分类	338
TrafficSign	11	重分类并补漏标	20
交通控制目标总数	414	删除 92 / 重分类 6 / 新增 36	358

同一模型，同一 150 张图像	复核前	复核后	变化
总体 Precision	0.808	0.736	-0.072
总体 Recall	0.536	0.611	+0.076
总体 mAP50	0.640	0.645	+0.006
总体 mAP50-95	0.469	0.475	+0.006
TrafficLight mAP50	0.734	0.774	+0.040
TrafficSign mAP50	0.338	0.320	-0.018

结果解读：复核后总体 Recall、mAP50 和 mAP50-95 均提高，TrafficLight mAP50 由 0.734 提高到 0.774。TrafficSign 的测试目标由 11 个增至 20 个，mAP50 由 0.338 小幅降至 0.320，说明旧结果因漏标而偏乐观。该下降不是优化失败，而是更完整标签对模型能力给出的更严格估计。

本次只提高独立测试结果的可信度，没有把测试集用于训练，也没有声称 1000 张完整数据集已经全部人工标注。复核清单、纠错配置、操作日志和复核后标签均随提交包保存。

十一、问题 6：多后端推理速度优化

在同一台笔记本上，以 150 张预加载测试图像、batch=1、640 像素输入和 20 次预热进行测量。计时包含模型预处理、推理及 NMS/后处理，排除磁盘解码；GPU 测试在计时前后显式同步 CUDA。

后端	设备	平均延迟(ms)	P95(ms)	FPS
PyTorch CPU	cpu	18.97	22.30	52.71
ONNX CPU	cpu	23.94	25.72	41.77
PyTorch GPU	0	5.18	5.83	192.98
ONNX GPU	0	5.65	7.47	176.84
TensorRT GPU FP16	0	3.12	3.72	320.63
双模型融合 CPU	cpu	34.35	38.35	29.11
双模型融合 GPU	0	9.65	10.35	103.60

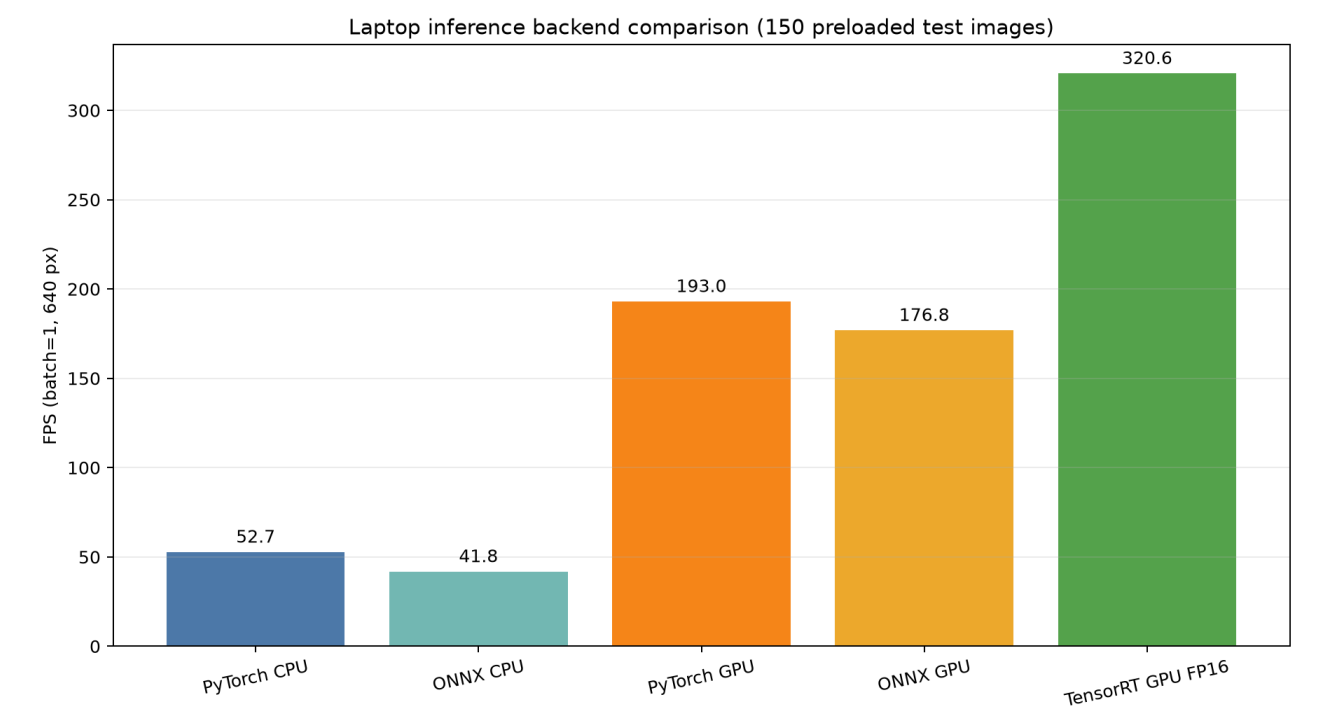


图 5 单模型 PyTorch、ONNX 与 TensorRT 笔记本推理速度对比

TensorRT GPU FP16 为 3.12 ms / 320.63 FPS，相对 PyTorch GPU 的 5.18 ms / 192.98 FPS 加速 1.66 倍。ONNX GPU 为 176.84 FPS，本机上未快于 PyTorch GPU；ONNX CPU 也未快于 PyTorch CPU，因此没有选择性隐藏负结果。双模型融合由 CPU 29.11 FPS 提升到 GPU 103.60 FPS，约加速 3.56 倍。

这些数据只反映笔记本软件推理管线。它们不包含真实摄像头采集、图像传输、车辆总线、规划控制和执行器响应，不能替代真实车载计算平台的摄像头到控制输出端到端延迟。

十二、演示与可复现产物

项目	结果
CPU 测试图像	150
输入尺寸	640
双模型 CPU 平均延迟	34.35 ms/帧
双模型 CPU 中位延迟	33.93 ms/帧
双模型 CPU FPS	29.11
演示视频	1000 帧, 15 FPS, 66.67 秒
视频编码与检查	H264, 1280x720, 1000/1000 帧解码通过

更新后的预加载基准显示，双模型融合在 CPU 上为 29.11 FPS、GPU 上为 103.60 FPS；旧版 6.45 FPS 包含磁盘读取和首次运行等额外开销，不能与本次纯软件推理口径直接混用。

主要提交产物

目录/文件	内容
weights/	道路参与者与交通控制两个模型的 best/last 权重
training/	最终 40 轮 args、results.csv 和训练曲线
evaluation/	融合 PR/P/R/F1 曲线和两份混淆矩阵
reports/	数据、CUDA、评估、FPS、视频及提升摘要
demo/	最终融合系统 66.67 秒 H.264 演示视频

十三、当前局限性

局限	当前影响	后续方向
完整训练集仍含伪标注	本次仅人工复核 150 张独立 test；不能代表 1000 张均为人工真值	继续分批复核训练/验证集或以原生标签重采
交通控制为远距离小目标	TrafficLight/Sign mAP50-95 较低	提高有效像素尺寸、补采近距离和类别多样性
双模型串行融合	虽可在 GPU 加速，但仍比单模型耗时	并行执行、知识蒸馏为单模型，并在目标芯片重新部署
无真实汽车计算平台	无法给出真实车载端到端延迟	在目标车载芯片上测摄像头输入到控制输出
地图数量仍有限	真实城市泛化能力未知	继续增加地图、真实天气、传感器噪声与域随机化

十四、实验结论

本次完善对旧报告问题 1、3、4、5 给出了可验证修复：稳定配置完成 40 轮；TrafficSign 唯一实例增至 1156；相邻帧泄漏通过严格分组和哈希检查消除；地图扩展到 Town05 与 Town10HD_Opt，并增加多天气、光照和随机种子。

最终类别专长融合在严格独立 test 上达到 Precision=0.880、Recall=0.590、mAP50=0.704、mAP50-95=0.403。相同评估口径下 mAP50 比旧模型提高 0.231。

问题 2 已对 150 张独立测试图像完成可追溯人工复核，使评估标签更完整、结论更可信；问题 6 已完成 CPU、GPU、ONNX 与 TensorRT 的同口径笔记本基准，TensorRT FP16 相对 PyTorch GPU 加速 1.66 倍。仍需如实保留两项边界：完整训练集尚未全部替换为人工真值，且尚无真实车载平台端到端延迟。

代码仓库：<https://github.com/xuzihao723/xiaomi-auto-drive>